Free Claude Code Plugin Brings Amazon's PR/FAQ Process to Engineers and Founders

punt-labs releases prfaq, an open-source tool that guides product discovery inside the terminal, producing a professionally typeset decision document in under an hour

Press Release

SAN FRANCISCO — Today, punt-labs released `prfaq`, a free, open-source Claude Code plugin that guides builders and founders through Amazon's Working Backwards PR/FAQ process to evaluate product ideas before building them. Unlike blog posts, templates, or workshops that sit outside the builder's workflow (see FAQ 12), `prfaq` runs as a slash command inside the same Claude Code session where the user already works. Builders who ship products at unprecedented speed can now apply the same rigor to deciding *which* products to ship.

Problem

AI coding tools have made it possible for a single person to build in a weekend what used to take a team a quarter. The bottleneck has shifted: the hard part is no longer building — it is knowing what to build and why (see FAQ 11). Builders and founders using Claude Code, Cursor, and similar tools ship fast, but most lack formal product training. They skip customer definition, skip competitive analysis, skip unit economics — and discover after burning through tokens and weeks of building that the product does not resonate.

The consequences are visible at both ends of the market. Analysts at HFS Research estimate the Global 2000 carry $1.5–2 trillion in accumulated technical debt [1]. At the startup end, AI-generated code introduces its own quality burden: 66% of developers cite "almost-right" AI output as their top frustration, and 45% report that debugging AI-generated code takes longer than expected [2]. Many products built during the current vibe coding wave already face significant rebuilds — not because the code is bad, but because the code solves the wrong problem with confidence.

Today, a builder who wants product discipline has three options: hire a product manager they cannot afford, read a book and try to apply frameworks manually, or just iterate — ship something, get feedback, ship again. Iteration works. Some of the best products started as something a builder made for themselves, and others appreciated. But iteration has real costs: every cycle burns tokens, burns calendar time marketing to early users, and burns the founder's energy. Getting customer feedback is often the actual bottleneck. Plenty of builders would develop stronger product sense if the frameworks were more accessible — available in the workflow where they already build, not in a book or a workshop they will never attend.

Solution

`prfaq` brings product thinking directly into the builder's environment. By typing `/prfaq` in any Claude Code session, the user starts a structured conversation: Claude asks about the target customer, the problem, the competitive landscape, and the business model. It asks hard questions — the kind a product manager would ask before greenlighting a project.

From these answers, Claude generates a complete PR/FAQ document: a one-page mock press release describing the product as if it has already shipped, followed by external FAQs (what a customer would ask) and internal FAQs (what leadership, finance, and engineering would ask).

The document is evaluated against Marty Cagan's four risks framework [3] — value, usability, feasibility, and viability — producing an honest assessment of whether the product is worth building.

The output is a LaTeX-compiled PDF (see FAQ 6): a clean, professional artifact suitable for sharing with co-founders, investors, or advisors. It is not a disposable brainstorm. It is a decision-making document designed to be read, debated, and revised.

Customer Quote

> *"I kept building MVPs that nobody used. I'd spend three weeks coding, show it to people, and get polite shrugs.* `prfaq` *made me realize I was solving my own problem, not my customer's. The PR/FAQ took forty-five minutes and saved me a month of wasted work. The hardest question it asked was 'what evidence do you have that customers want this?' I didn't have any. That was the answer."*
>
> — **Marcus Okonkwo**, Founder, SyncPilot (field-service scheduling)

Getting Started

Getting started with `prfaq` takes three steps. First, run the one-line installer: `curl -fsSL https://punt-labs.github.io/prfaq/install | bash`. This clones the plugin repository and registers it with Claude Code automatically. Second, open any Claude Code session and type `/prfaq`. Third, answer the discovery questions — Claude guides the conversation, asks follow-ups, and drafts the PR/FAQ document. Within an hour, the user has a compiled PDF that forces clarity on customer, problem, differentiation, and business model. No account required, no subscription, no configuration.

Spokesperson Quote

> "The people building products with AI are making product decisions every day — they just don't always have the frameworks. We built `prfaq` because the builders using Claude Code are also the ones deciding what to build and for whom (see FAQ 10), and they deserve a tool that meets them where they already work. The PR/FAQ process has been proven at Amazon for two decades [4]. We didn't invent it. We made it accessible inside a terminal."
>
> — **Jason Freeman**, Founder, punt-labs

Call to Action

`prfaq` is free and open source, available now at https://github.com/punt-labs/prfaq. Install it, type `/prfaq`, and find out if your next product idea is worth building before you build it.

Frequently Asked Questions

External FAQs

Q1: What is prfaq and who is it for?

`prfaq` is a free Claude Code plugin that guides engineers and founders through the Amazon Working Backwards PR/FAQ process. The primary audience is technical builders who use AI coding tools to create products and want to validate an idea before committing weeks or months of development time. As Claude Code's user base expands beyond professional engineers [5], features like `/prfaq:import` — which enriches an existing document rather than generating one from scratch — extend the plugin's reach to non-technical users who adopt Claude Code as a thinking partner.

Q2: How is this different from using a PR/FAQ template?

A template is a blank page with headings. `prfaq` is an interactive process: Claude asks discovery questions, challenges weak answers, identifies gaps in customer evidence, and generates the document from your responses. It applies the four risks framework to evaluate your idea honestly, not just format it. A template tells you *what* to write. `prfaq` helps you figure out *what to think*.

Q3: How do I get started?

Run the one-line installer (`curl -fsSL https://punt-labs.github.io/prfaq/install | bash`), then type `/prfaq` in any Claude Code session. The plugin walks you through the entire process. No account, no API keys, no configuration. Installation takes under a minute.

Q4: What does the output look like?

A professionally typeset PDF compiled from LaTeX. The document includes a one-page press release, external and internal FAQs, and a four-risks assessment. It is designed to be shared with co-founders, investors, or advisors — not a throwaway brainstorm document.

Q5: Do I need to know LaTeX?

No. The plugin generates the LaTeX source and compiles it automatically. You never see or edit LaTeX unless you choose to. You need a TeX distribution installed (TeX Live or similar), which the installer checks for and provides setup instructions if missing.

Q6: Why LaTeX instead of Markdown?

Three reasons. First, a PR/FAQ is a decision document — it gets shared with co-founders, investors, and advisors. LaTeX produces professionally typeset PDFs with consistent typography, precise layout, and visual credibility that Markdown-to-PDF pipelines do not match. The medium signals that the thinking is serious. Second, LaTeX's structured environments (`faqpair`, `customerquote`, `spokespersonquote`) give the LLM a precise grammar to write into — the AI reads and writes `.tex`, the human reads the compiled `.pdf`. This separation means the source format can be rich and semantic without burdening the author. Third, LaTeX's package ecosystem (biblatex for citations, mdframed for callout boxes, enumitem for structured lists) provides extensibility that Markdown lacks without custom renderers.

The trade-off is friction: LaTeX requires a TeX distribution (~4 GB), and users unfamiliar with it may find compilation errors intimidating. The installer mitigates this with automated setup, and the plugin handles all LaTeX generation — the user never writes `.tex` directly. This is a two-way door: if user feedback shows the TeX dependency is a meaningful adoption barrier, adding a Markdown output mode is straightforward since the content generation is format-independent. The structured thinking matters more than the output format.

Q7: What happens to my data?

Everything runs locally. The plugin is a set of prompt files and a LaTeX template — it does not phone home, collect telemetry, or store your data anywhere. Your PR/FAQ content is processed by Claude Code using your existing Anthropic API relationship. The generated `.tex` file and compiled PDF stay on your machine. Data handling is governed entirely by Claude Code's existing privacy model and your Anthropic terms of service.

Q8: How much does it cost?

Free. `prfaq` is open source under a permissive license. There is no paid tier, no freemium upsell, and no planned monetization. The plugin itself adds zero cost. You do need an active Claude Code subscription or API access, which is a separate cost from Anthropic.

For API users, the underlying token cost is small. A full PR/FAQ session — discovery conversation, document generation, and peer review — cumulatively consumes roughly 100–200K input tokens and 20–40K output tokens across multiple turns (context is re-sent each turn). At Sonnet pricing ($3/$15 per million tokens), that is approximately $0.60–1.20 per initial draft. Each feedback cycle is lighter: roughly 40–80K cumulative input tokens and 10–20K output tokens, or about $0.25–0.55 per round. A typical document through three or four revision cycles costs under $4 in total token spend. Opus sessions cost roughly 1.7× more.

For comparison, a product consultant charges $150–300 per hour and a PR/FAQ workshop runs $2,000–5,000. Claude Code subscribers on the Pro plan ($20/month), Max 5× ($100/month), or Max 20× ($200/month) pay nothing incremental — the token usage falls within the subscription's included allowance.

Q9: Can I iterate on the document?

Yes. The plugin generates a `.tex` file in your project directory. You can re-run the process, edit sections, or ask Claude to revise specific parts. The Working Backwards process is designed for rapid iteration — the point is to iterate on a one-page document instead of iterating on a shipped product.

Internal FAQs

Value & Market

Q10: What is the total addressable market?

The primary market is engineers and technical founders who use AI coding tools to create products. As of early 2026, 84% of professional developers use or plan to use AI tools in their workflows, with 51% relying on them daily [2]. Claude Code's trajectory — surpassing $2.5B in annualized revenue [6] and doubling since January — indicates a large and growing base, though exact user counts are not publicly available. The subset who make product decisions — founders, indie hackers, startup engineers, and technical co-founders — is the primary audience. A secondary audience is emerging. Claude Code is increasingly adopted by non-programmers: a *New York Times* profile documented a school administrator, photographer, prosecutor, finance professor, and welding business owner all building functional tools with Claude Code and no prior coding experience [5]. As this crossover accelerates, features like `/prfaq:import` extend the plugin's reach to users who adopt Claude Code as a strategic thinking partner, not just a code generator. This is not a TAM-based revenue argument (the plugin is free); it is a reach argument. The primary audience is technical builders today, with organic growth into the broader Claude Code user base over time.

Q11: What evidence do we have that customers want this?

Three categories of indirect evidence, but no primary customer data yet. First, the "vibe coding" phenomenon: thousands of products are being built at speed without product discipline, and rebuilds are already visible across startup communities and forums. Second, the explosion of "how to think about product" content targeting builders — books, courses, and newsletters suggest unmet demand for product skills among people who build with AI tools but lack formal product training. Third, the Working Backwards process itself has two decades of validation at Amazon [4], where it is mandatory for new product proposals. The gap is not whether the framework works — it is accessibility. Builders do not attend PM workshops. They install plugins. *Note: we have not yet validated that technical builders want product frameworks delivered inside Claude Code. Five to ten customer discovery interviews with engineers and technical founders would significantly strengthen this evidence base. As the non-technical audience grows [5], a separate round of interviews with that segment would inform whether* `/prfaq:import` *resonates.*

Q12: Who are the competitors and why will we win?

The nearest competitor is not a product management tool — it is the user pasting a PR/FAQ template into Claude.ai or any AI chat and having a free-form conversation. This works for experienced practitioners who already know the framework. What it lacks: structured reference guides that catch common mistakes, the four risks evaluation framework, automated peer review against cognitive biases, LaTeX compilation to a shareable PDF, and a repeatable process that improves across uses. The plugin encodes twenty years of Working Backwards practice into a reusable workflow; a blank chat starts from zero every time.

Beyond that, traditional competitors include product management tools (Productboard, Aha!, Notion templates) and business planning tools (Lean Canvas generators). None of these are integrated into the builder's working environment. They all require context-switching: leave the current tool, open a browser, fill in a form. A Claude Code plugin runs in the same session where the builder already works — the shift from "building" to "product thinking" happens in a single command, not a tab switch.

Technical

Q13: What are the major technical risks?

The technical risks are minimal. The plugin is a set of markdown files (skill definition and reference guides) plus a LaTeX template and a shell script. It requires no backend, no API, no database, no infrastructure. The primary technical dependency is the Claude Code plugin system, which is stable and well-documented. The LaTeX dependency requires users to have a TeX distribution installed, which is a mild friction point for users who do not already have one. Mitigation: the installer checks for TeX and provides platform-specific setup instructions.

Q14: What dependencies exist on other teams or systems?

The plugin depends on Anthropic's Claude Code plugin system for loading and execution, and on a local TeX distribution for PDF compilation. Both are external dependencies outside our control. The Claude Code plugin API is stable (v1), and TeX Live is a mature, 30-year-old ecosystem. If Anthropic changes the plugin format, the migration would be straightforward — the plugin is a handful of files with simple structure. If TeX Live is unavailable, the plugin still generates the `.tex` source file; only PDF compilation is affected.

Q15: If this succeeds, what breaks first?

Nothing at the infrastructure level — the plugin is stateless, runs locally, and requires no servers. At scale, the pressures are human, not technical: (1) GitHub issue volume — at 1,000+ users, support requests and feature demands will exceed solo maintainer capacity; mitigation is clear contribution guidelines and aggressive scope discipline (see Won't Do list); (2) prompt maintenance — as Claude models evolve, skill prompts may need tuning to maintain output quality; (3) backwards compatibility — if Anthropic changes the plugin system, the plugin must migrate; the simple file-based architecture makes this low-risk; (4) community expectations — a popular free tool attracts requests for features that contradict the design philosophy (dashboards, collaboration, other LLM support). The Won't Do list exists to make these decisions in advance, not under pressure.

Q16: What is the estimated development timeline?

The plugin is built. Initial development took one session. Ongoing work is iterative: improving the skill prompts based on real usage, expanding reference material, and potentially adding alternative output formats. There is no multi-month roadmap because there is no infrastructure to build.

Business

Q17: What is the revenue model?

There is none. `prfaq` is free and open source under a permissive license. The strategic value is threefold: (1) it is a proving ground for the Claude Code plugin development pattern, informing future commercial plugins; (2) it builds reputation and visibility in the Claude Code ecosystem; (3) it creates a feedback loop for understanding how builders approach product thinking, which informs other products at punt-labs.

Q18: What does the cost structure look like at steady state?

`prfaq` has zero infrastructure cost — no servers, no databases, no APIs. The real cost is maintainer time. At current scale (pre-launch): approximately 2 hours per week on development and prompt refinement. At the 500-star target: an estimated 4–8 hours per week covering issue triage, community engagement, prompt maintenance as Claude models evolve, and occasional LaTeX template updates. At 1,000+ users: maintainer capacity becomes the constraint. At that point, the project either recruits community co-maintainers or narrows support scope to bug fixes only. The opportunity cost is hours not spent on other punt-labs projects. This is acceptable while the plugin serves as a learning vehicle for Claude Code plugin development; it should be re-evaluated if other punt-labs priorities require the same hours.

Q19: What are the key metrics for success?

Three metrics, measured via GitHub analytics and community signals:

1. **GitHub stars and forks** — target: 500 stars within 6 months. This measures discoverability and perceived value.
2. **Issues and pull requests from external contributors** — target: 10 external PRs within 6 months. This measures whether the plugin is useful enough that people invest time improving it.
3. **Mentions in builder and developer communities** (Hacker News, Twitter/X, Reddit, Discord servers) — target: 3 organic mentions within 3 months. This measures whether the plugin solves a real problem that people tell others about.

If none of these targets are met after 6 months, the plugin is not reaching its audience and the distribution strategy should be reconsidered.

Q20: Why now? What has changed?

Three shifts converged. First, AI coding tools reached mainstream adoption in 2025–2026, creating a large population of engineers and technical founders who can build products independently but lack product discipline. Second, Claude Code's plugin system matured, making it possible to deliver structured workflows inside the developer's existing tool. Third, the consequences of "vibe coding without product thinking" are becoming visible — failed launches, rebuilds, wasted months — creating demand for lightweight product frameworks that do not require hiring a PM or leaving the terminal. A fourth tailwind is emerging: Claude Code is crossing over to non-technical users [5], which expands the potential audience beyond the initial technical core.

Q21: What is the customer acquisition strategy?

Distribution is the product strategy for a free plugin. Primary channels: (1) GitHub discoverability — README, topics, and stars drive organic search traffic; (2) community seeding — posts on Hacker News, Reddit r/ClaudeAI, and Twitter/X targeting the "vibe coding" and AI-builder audience; (3) content marketing — the plugin's own PR/FAQ serves as a case study and demo artifact; (4) word-of-mouth — builders who find the plugin useful share the PDF output, which includes a link back to the repository. There is no paid acquisition.

Estimated time investment: 2–4 hours per week on community engagement for the first 3 months. Conversion assumption: a front-page Hacker News post reaches approximately 20–30K views and converts at roughly 0.1–0.2%, yielding around 20–60 installs per post. These numbers are testable hypotheses, not forecasts. If organic channels do not work within 6 months, the distribution hypothesis is wrong and should be revisited.

Q22: What are we not building?

We are not building a SaaS product management platform. We are not building a competitor to Jira, Linear, Productboard, or Notion. We are not adding collaboration features, dashboards, or analytics. We are not building a web interface. The plugin is a single-player tool that runs locally, produces a document, and gets out of the way. Scope discipline is the feature. See the Feature Appendix for the full Must Do / Should Do / Won't Do breakdown.

Q23: It is one year from now and prfaq failed. What went wrong?

Most likely failure scenario: builders do not want product discipline, even when it is free and frictionless. The vibe coding audience builds for the joy of building, and a tool that asks "what evidence do you have that customers want this?" feels like a buzzkill, not a feature. Second scenario: the plugin does not surface in a crowded marketplace — the Claude Code plugin ecosystem is growing rapidly, and discoverability is poor. If users cannot find `prfaq` among hundreds of alternatives, organic adoption stalls regardless of quality. Third scenario: the output quality is not high enough to share with stakeholders — if the generated PR/FAQ reads like AI slop rather than a credible decision document, users will not trust it for real decisions. Fourth scenario: the PR/FAQ process is too heavyweight for the target audience — builders want a quick sanity check, not a formal document, and a lighter-weight alternative captures the market.

Risk Assessment

Risk	Rating	Assessment
Value	Medium	Market-level evidence strong (proven framework, large technical audience). Customer-level evidence absent — no interviews confirming demand for this delivery mechanism from engineers and technical founders. Plugin is free, so adoption barrier is low.
Usability	Low	One-line install, one slash command, Claude guides interactively. TeX dependency is the main friction; installer mitigates. Time to value under one hour.
Feasibility	Low	Built and functional. No backend, no infrastructure. Hard problem (interactive AI guidance) solved by Claude itself.
Viability	Medium	Zero infrastructure cost but maintainer time is the binding constraint. At 500+ stars, estimated 4–8 hrs/week on issues, community, and prompt maintenance. Mitigation: contribution guidelines, scope discipline, community co-maintainers at scale.

Feature Appendix

Must Do (V1)

- Interactive discovery workflow — structured questions that guide the user from idea to document
- Complete PR/FAQ document generation — press release, external FAQs, internal FAQs, four risks assessment
- LaTeX compilation to professional PDF — shareable artifact, not a disposable brainstorm
- Reference guides — encoded best practices for PR structure, FAQ structure, four risks, and common mistakes
- Revise mode — surgical edits to an existing document without regenerating from scratch
- Peer review agent — automated critique against Working Backwards principles and cognitive bias checklist

Done (Shipped in v0.4.0)

- Research sub-agent — scans local `./research/` directory, quarry-mcp indexes, and the web for evidence
- Biblatex citation system — academic-style citations for claims, linking evidence to assertions
- Unit economics reference guide — structured framework for viability risk evaluation
- Principal engineer reference guide — feasibility risk: architecture trade-offs, irreversible decisions, operational complexity
- UX bar raiser reference guide — usability risk: customer journey, cognitive load, mental model alignment
- Judgment-to-FAQ cross-referencing — `\pageref{faq:...}` links from press release to FAQ justifications

Should Do (Future Iterations)

- Markdown output mode — alternative to LaTeX for users without a TeX distribution
- Comparative analysis mode — evaluate two competing product ideas side-by-side
- Template customization — allow users to override the LaTeX template for branding

Won't Do

- **Web dashboard or SaaS platform** — the plugin runs locally inside Claude Code. Adding a web layer would introduce infrastructure, authentication, and hosting costs that contradict the zero-ops design. If users want to share documents, they share the PDF.
- **Collaboration or multi-user editing** — PR/FAQ is a single-author process by design. The document is written by one person, then reviewed by others. Real-time collaboration would encourage design-by-committee, which the Working Backwards process explicitly avoids.
- **Project management features** — no task tracking, no roadmap views, no sprint planning. The plugin produces a decision document, not a project plan. Users who need project management have Linear, Jira, or beads.
- **Alternative LLM backends** — the plugin is a Claude Code plugin. Supporting OpenAI, Gemini, or local models would fragment the prompt engineering, dilute quality, and multiply the testing surface for no clear user benefit.

References

[1] HFS Research and Publicis Sapient. *Smash Through Tech Debt: Why AI Is the Jackhammer*. Survey of 608 IT and business leaders estimating Global 2000 accumulated technical debt at $1.5–2 trillion. 2025. URL: https://www.hfsresearch.com/research/publicis-sapient-jackhammer/.

[2] Stack Overflow. *2025 Developer Survey*. Annual survey of 65,000+ developers on tools, practices, and AI adoption. 2025. URL: https://survey.stackoverflow.co/2025.

[3] Marty Cagan. *Inspired: How to Create Tech Products Customers Love*. 2nd ed. Wiley, 2018. ISBN: 978-1119387503.

[4] Colin Bryar and Bill Carr. *Working Backwards: Insights, Stories, and Secrets from Inside Amazon*. St. Martin's Press, 2021. ISBN: 978-1250267597.

[5] Natallie Rocha. "This A.I. Tool Is Going Viral. Five Ways People Are Using It." In: *The New York Times* (Jan. 2026). Profiles five non-programmers using Claude Code: school administrator, photographer, prosecutor, finance professor, welding business owner. URL: https://www.nytimes.com/2026/01/23/technology/claude-code.html.

[6] Constellation Research. *Anthropic's Claude Code Revenue Doubled Since Jan. 1*. Claude Code annualized run-rate revenue reached approximately $2.5B as of February 2026, more than doubling since January 1. Feb. 2026. URL: https://www.constellationr.com/insights/news/anthropics-claude-code-revenue-doubled-jan-1.